Smoothie Editing: space_battle.blend • 116.3 MB • *(modified 1 hr. ago)* • 3/27/2026, 10:25:57 PM

You

Download some free spaceship models from blenderkit and make them fly around. Make a colorful robot on the ground that fires bullets at the spaceship. The animation should be 10 seconds long.

Smoothie

Let me start by checking the scene and BlenderKit availability, then find some spaceships!

ToolSearch

Reading scene

Checking BlenderKit

Reading timeline

Smoothie

BlenderKit is ready to go! Let me search for some spaceships.

Searching BlenderKit

Searching BlenderKit

Smoothie

Found some great free spaceships! Let me import 3 of them at different positions. I'll grab the Kezrek G1 Spaceship, Phantom Starship, and the Sci Fi Spacecraft.

Importing from BlenderKit

Importing from BlenderKit

Importing from BlenderKit

Smoothie

All three spaceships are downloading! Let me check the scene to see their names once they arrive.

Reading scene

Smoothie

I can see 2 of the 3 ships loaded — the Phantom Starship might still be downloading. Let me

check once more and also look at the hierarchies.

🔍 Searching objects

📝 Reading hierarchy

📝 Reading hierarchy

⚙ ToolSearch

📝 Reading hierarchy

📝 Reading hierarchy

Smoothie

The Kezrek G1 Spaceship is standalone, and the Sci Fi Spacecraft is a parent empty with child parts. The Phantom Starship may still be downloading. Let me proceed — I'll build this scene step by step:

1. **First** — Clean up, set timeline to 10 seconds, build the ground and colorful robot
2. **Then** — Animate the spaceships flying around
3. **Finally** — Animate the robot firing bullets at them

Let's start building the robot and scene!

1 🔨 Code generated 7162 bytes

✓ Executed See Appendix #1

Smoothie

The robot is built! Now let me check if the Phantom Starship has loaded, then animate the spaceships flying around.

🔍 Searching objects

🔍 Searching objects

Smoothie

All 3 spaceships are loaded! Now let me animate them flying around in different flight paths, and make the robot rotate to track them.

2 🔨 Code generated 3786 bytes

✓ Executed See Appendix #2

Smoothie

Now let me create the bullet-firing animation! I'll spawn glowing bullets from the robot's gun that fly toward the spaceships.

3 🔨 Code generated 4193 bytes

✓ Executed See Appendix #3

Smoothie

Excellent! Now let me add some finishing touches — a muzzle flash effect on the robot when it fires, and update the camera to frame the action nicely.

4 🔨 Code generated 2748 bytes

✓ Executed See Appendix #4

Smoothie

Now let me save the project notes for future reference.

📝 Updating project notes

Smoothie

Your space battle scene is ready! 🚀🤖 Here's what I built:

3 Spaceships (downloaded from BlenderKit):

- **Kezrek G1 Spaceship** — flies in a circular orbit with smooth bobbing
- **Sci Fi Spacecraft** — weaves through a figure-8 flight path
- **Phantom Starship** — takes a wild, swooping complex orbit

Colorful Robot on the ground with:

- **Blue** body, **red** head, **green** arms, **orange** legs, **purple** antenna
- Glowing **yellow eyes** and a pulsing **magenta antenna ball**
- A **gun arm** that fires 10 glowing orange bullets at the nearest ship
- **Muzzle flash** effect that lights up with each shot
- The whole robot **rotates to track** whichever ship is closest

Scene touches:

- Dark night sky background
- Warm sun + cool blue fill lighting

- Camera slowly pans around the action over the 10 seconds

Hit **Play** (Spacebar or the play button) to watch it in action! The animation runs from frame 1–240 (10 seconds at 24fps).

You

Make the animation longer - 20 seconds. Add several copies of each spaceship, make all the spaceships fly higher in interesting patterns (circles, figure eights, evasive maneuvers etc). Make the robot's gun fire streams of bullets (10 bullets in quick succession), and add more shooting rounds. Make some of the shots miss but two of the spaceships get hit and then dive to the ground while spinning.

Smoothie

Big upgrade! Let me plan this out and build it in stages. First, let me extend the timeline, duplicate spaceships, and set up all the new flight paths.

ToolSearch

Smoothie

This is a big overhaul — I'll build it in stages. First, let me clear old animations, extend the timeline, duplicate the spaceships, and set up all the new flight paths including the two crash sequences.

5 🔨 Code generated 2912 bytes

✓ Executed See Appendix #5

Smoothie

Ships duplicated! Now let me animate all 7 ships with unique flight paths — and make 2 of them get hit and crash.

6 🔨 Code generated 10690 bytes

✓ Executed See Appendix #6

Smoothie

All ships are flying. Now let me create the bullet stream system — 10 rapid-fire bullets per burst, multiple bursts, with the two critical hits!

7 🔨 Code generated 5516 bytes

✓ Executed See Appendix #7

Smoothie

96 bullets across 10 bursts! Now let me add the muzzle flash effects, antenna pulse, and update the camera to capture all the action over 20 seconds.

8 🔨 Code generated 3654 bytes

✓ Executed See Appendix #8

Smoothie

Now let me update the project notes to reflect all the changes.

📝 Updating project notes

Smoothie

Your upgraded space battle is ready! Here's what changed:

🚀 **7 Spaceships (up from 3)**

- **3 Kezrek G1s** — wide circle, tight counter-circle, and elliptical orbit
- **2 Sci Fi Spacecraft** — figure-8 and wild evasive zigzag
- **2 Phantom Starships** — complex swooping and fast weaving
- All flying much higher (18–35 altitude) with varied, interesting paths

🔫 **Bullet Streams (96 bullets!)**

- **10 bursts** throughout the 20 seconds, each firing **10 rapid bullets** (3 frames apart)
- Most shots have realistic spread and **miss** — bullets streak past the ships
- **Two direct hits**: Burst at frame 270 nails **Kezrek_Copy3**, burst at frame 390 tags **Phantom Starship**

💥 **Crash Sequences**

- Both hit ships **spin wildly and tumble to the ground** over ~80 frames, wobbling and spiraling as they fall
- Camera dynamically **pushes in closer** during crash moments for drama

✨ **Effects**

- **Rapid muzzle flash** strobes with each bullet in a burst
- **Antenna pulses faster** when the robot is actively firing
- **Cinematic camera** pans 120° around the scene with dynamic zoom during crashes

Hit **Play** to watch the full 20-second battle! 🤖⚡

You

It's hard to see the bullets, make them bigger. Make the terrain square bigger. Add a dark sky with twinkling stars

ToolSearch

Smoothie

Let me make those three changes — bigger bullets, larger terrain, and a starry sky!

9 🔨 Code generated 5808 bytes

Pending See Appendix #9

10 🔨 Code generated 4449 bytes

✓ Executed See Appendix #10

Smoothie

Done! Here's what changed:

- **Bullets** are now **3x bigger** with a brighter **yellow-hot glow** (emission cranked to 30) — much easier to see streaking through the sky
- **Terrain** expanded from 100×100 to **300×300** — no more seeing the edges
- **Starry sky** — deep dark blue-black background filled with scattered twinkling stars that subtly shimmer over the 20-second animation

Hit play and enjoy the night battle! 🌌🤖💥

You

The stars appear initially but disappear almost immediately after the animation begins.

Smoothie

The problem is the animated mapping offset is shifting the entire noise pattern, moving stars past the sharp threshold so they vanish. Let me fix it — static star positions with a subtle separate brightness twinkle.

11 🔨 Code generated 3730 bytes

✓ Executed See Appendix #11

Smoothie

Fixed! The issue was that the animation was shifting the *star positions* themselves, pushing them past the visibility threshold. Now:

- **Star positions are completely static** — they stay put the entire animation
- **A separate twinkle layer** gently shifts which stars are brighter or dimmer over time, but never below 40% brightness — so no stars disappear
- The effect is a subtle, natural shimmer across the sky ✨

Hit play — the stars should stay visible throughout all 20 seconds now!

You

It's still happening - initially there are many stars, most of them disappear very quickly

Smoothie

The issue is likely the texture coordinate type — "Object" coordinates in a world shader can behave unpredictably. Let me rebuild this properly using stable coordinates and no animation on the star layer at all.

12 🔨 Code generated 2196 bytes

✓ Executed See Appendix #12

Smoothie

Try playing the animation now — the stars are built with fully static `Generated` coordinates and zero animation on the world shader. They should stay locked in place throughout all 480 frames. Let me know if it's working!

You

Maybe it's just that I'm in preview mode, but it doesn't look like I expect. Will it look better when I render with a good rendering engine?

⚙ ToolSearch

⚙ Reading render settings

Smoothie

Yes, that's a big part of it! You're on **EEVEE** right now and the viewport preview doesn't show emission glow properly. A proper render will look much better. Let me set up some things to make it really pop — I'll add **bloom/glare** via the compositor so all the bullet emissions and star glow really shine, and bump up the EEVEE settings.

13 🔨 Code generated 1670 bytes
Pending See Appendix #13

14 🔨 Code generated 1579 bytes
Pending See Appendix #14

15 🔨 Code generated 589 bytes
✓ Executed See Appendix #15

Smoothie

Got it — Blender 5.1 uses `compositing_node_group` instead of `node_tree`. Let me fix that.

16 🔨 Code generated 1709 bytes
Pending See Appendix #16

17 🔨 Code generated 550 bytes
✓ Executed See Appendix #17

18 🔨 Code generated 515 bytes
✓ Executed See Appendix #18

Smoothie

Now I see — Blender 5.1 moved all Glare settings to node inputs. Let me set it up properly.

19 🔨 Code generated 729 bytes
✓ Executed See Appendix #19

20 🔨 Code generated 437 bytes

✓ Executed See Appendix #20

21 🔨 Code generated 779 bytes

✓ Executed See Appendix #21

Smoothie

Now I know the exact API. Let me set it all up properly!

22 🔨 Code generated 1503 bytes

Pending See Appendix #22

23 🔨 Code generated 611 bytes

✓ Executed See Appendix #23

24 🔨 Code generated 896 bytes

✓ Executed See Appendix #24

25 🔨 Code generated 813 bytes

✓ Executed See Appendix #25

Smoothie

Now I know the right node types. Let me wire it all up properly.

26 🔨 Code generated 1763 bytes

✓ Executed See Appendix #26

Smoothie

The compositor bloom is set up! Press **F12** to render a test frame — you should see a huge difference compared to the viewport preview:

- **Bullets** will have a hot orange glow halo
- **Muzzle flash** will bloom brightly
- **Robot eyes & antenna** will glow
- **Stars** will have soft shimmer against the dark sky

The viewport in solid/material mode doesn't show emission glow or compositor effects — that's why it looked flat before. The rendered result will be much more dramatic! 🔥

You

What's the available context window size for this chat? And how much of it have we filled so far?

Smoothie

I don't have direct access to see my context window usage metrics. I can't introspect on how many tokens we've used or how much remains.

That said, here's what I can tell you:

- **My context window is 200K tokens** (Claude Opus 4)
- We've had a fairly lengthy conversation with multiple tool calls, scene reads, and several large code blocks, so we've likely used a **significant portion** — I'd estimate roughly 40-60% based on the volume of back-and-forth

If you're worried about running low, the good news is that the **project notes (smoothie.md)** capture all the key details about the scene — object names, animation structure, bullet system, etc. So if we ever need to start a fresh conversation, I can pick right back up by reading those notes.

Want to keep going with changes to the scene, or is there anything else you'd like to tweak?

Appendix — Generated Code

1 Code Block #1 7162 bytes ✓ Executed

```python

import bpy
import math
from mathutils import Vector

# --- Scene Setup ---
# Set timeline to 10 seconds at 24fps = 240 frames
scene = bpy.context.scene
scene.frame_start = 1
scene.frame_end = 240
scene.frame_current = 1

# Delete default cube
if "Cube" in bpy.data.objects:
    bpy.data.objects.remove(bpy.data.objects["Cube"], do_unlink=True)

# --- Ground Plane ---
bpy.ops.mesh.primitive_plane_add(size=100, location=(0, 0, 0))
ground = bpy.context.active_object
ground.name = "Ground"
mat_ground = bpy.data.materials.new("Ground_Mat")
mat_ground.use_nodes = True
bsdf = mat_ground.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.15, 0.2, 0.15, 1)  # Dark green
bsdf.inputs["Roughness"].default_value = 0.8
ground.data.materials.append(mat_ground)

# --- Helper: Create colored material ---
def make_mat(name, color, metallic=0.0, roughness=0.5, emission_color=None, emission_st
    mat = bpy.data.materials.new(name)
    mat.use_nodes = True
    bsdf = mat.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Base Color"].default_value = (*color, 1)
    bsdf.inputs["Metallic"].default_value = metallic
    bsdf.inputs["Roughness"].default_value = roughness
    if emission_color and emission_strength > 0:
        bsdf.inputs["Emission Color"].default_value = (*emission_color, 1)
        bsdf.inputs["Emission Strength"].default_value = emission_strength
    return mat

```

```
# Robot materials
mat_body = make_mat("Robot_Body", (0.0, 0.4, 0.9), metallic=0.8, roughness=0.3)       #
mat_head = make_mat("Robot_Head", (0.9, 0.1, 0.1), metallic=0.7, roughness=0.3)
mat_eye = make_mat("Robot_Eye", (1.0, 1.0, 0.0), emission_color=(1, 1, 0), emission_str
mat_arm = make_mat("Robot_Arm", (0.1, 0.8, 0.2), metallic=0.6, roughness=0.4)
mat_leg = make_mat("Robot_Leg", (0.9, 0.5, 0.0), metallic=0.6, roughness=0.4)
mat_antenna = make_mat("Robot_Antenna", (0.8, 0.0, 0.8), metallic=0.9, roughness=0.2)
mat_gun = make_mat("Robot_Gun", (0.3, 0.3, 0.3), metallic=0.9, roughness=0.2)
mat_muzzle = make_mat("Robot_Muzzle", (1.0, 0.2, 0.0), emission_color=(1, 0.3, 0), emis

# --- Build Robot ---
# All parts parented to a root empty for easy animation
bpy.ops.object.empty_add(type='PLAIN_AXES', location=(0, 0, 0))
robot_root = bpy.context.active_object
robot_root.name = "Robot_Root"

# Feet / base
bpy.ops.mesh.primitive_cube_add(size=1, location=(-0.5, 0, 0.3))
foot_l = bpy.context.active_object
foot_l.name = "Robot_FootL"
foot_l.scale = (0.4, 0.6, 0.3)
foot_l.data.materials.append(mat_leg)
foot_l.parent = robot_root

bpy.ops.mesh.primitive_cube_add(size=1, location=(0.5, 0, 0.3))
foot_r = bpy.context.active_object
foot_r.name = "Robot_FootR"
foot_r.scale = (0.4, 0.6, 0.3)
foot_r.data.materials.append(mat_leg)
foot_r.parent = robot_root

# Legs
bpy.ops.mesh.primitive_cylinder_add(radius=0.12, depth=1.0, location=(-0.5, 0, 1.0))
leg_l = bpy.context.active_object
leg_l.name = "Robot_LegL"
leg_l.data.materials.append(mat_leg)
leg_l.parent = robot_root

bpy.ops.mesh.primitive_cylinder_add(radius=0.12, depth=1.0, location=(0.5, 0, 1.0))
leg_r = bpy.context.active_object
leg_r.name = "Robot_LegR"
leg_r.data.materials.append(mat_leg)
leg_r.parent = robot_root

# Body (torso)
bpy.ops.mesh.primitive_cube_add(size=1, location=(0, 0, 2.0))
torso = bpy.context.active_object
```

```
torso.name = "Robot_Body"
torso.scale = (0.8, 0.5, 0.7)
torso.data.materials.append(mat_body)
torso.parent = robot_root

# Chest detail - a bright stripe
bpy.ops.mesh.primitive_cube_add(size=1, location=(0, -0.26, 2.0))
chest_stripe = bpy.context.active_object
chest_stripe.name = "Robot_ChestStripe"
chest_stripe.scale = (0.6, 0.02, 0.15)
mat_stripe = make_mat("Robot_Stripe", (1.0, 0.8, 0.0), metallic=0.5)
chest_stripe.data.materials.append(mat_stripe)
chest_stripe.parent = robot_root

# Neck
bpy.ops.mesh.primitive_cylinder_add(radius=0.1, depth=0.3, location=(0, 0, 2.5))
neck = bpy.context.active_object
neck.name = "Robot_Neck"
neck.data.materials.append(mat_body)
neck.parent = robot_root

# Head
bpy.ops.mesh.primitive_cube_add(size=1, location=(0, 0, 3.0))
head = bpy.context.active_object
head.name = "Robot_Head"
head.scale = (0.5, 0.4, 0.4)
head.data.materials.append(mat_head)
head.parent = robot_root

# Eyes (two glowing spheres)
for side, x in [("L", -0.18), ("R", 0.18)]:
    bpy.ops.mesh.primitive_uv_sphere_add(radius=0.08, location=(x, -0.2, 3.05))
    eye = bpy.context.active_object
    eye.name = f"Robot_Eye{side}"
    eye.data.materials.append(mat_eye)
    eye.parent = robot_root

# Antenna
bpy.ops.mesh.primitive_cylinder_add(radius=0.03, depth=0.5, location=(0, 0, 3.5))
antenna_pole = bpy.context.active_object
antenna_pole.name = "Robot_AntennaPole"
antenna_pole.data.materials.append(mat_antenna)
antenna_pole.parent = robot_root

bpy.ops.mesh.primitive_uv_sphere_add(radius=0.08, location=(0, 0, 3.8))
antenna_ball = bpy.context.active_object
antenna_ball.name = "Robot_AntennaBall"
```

```
mat_antenna_glow = make_mat("Robot_AntennaGlow", (0.8, 0, 0.8), emission_color=(1, 0, 1
antenna_ball.data.materials.append(mat_antenna_glow)
antenna_ball.parent = robot_root

# --- Arms ---
# Left arm
bpy.ops.mesh.primitive_cylinder_add(radius=0.1, depth=0.8, location=(-1.0, 0, 2.1))
arm_l = bpy.context.active_object
arm_l.name = "Robot_ArmL"
arm_l.rotation_euler = (0, 0, 0)
arm_l.data.materials.append(mat_arm)
arm_l.parent = robot_root

# Right arm - this is the gun arm!
# Upper arm
bpy.ops.mesh.primitive_cylinder_add(radius=0.1, depth=0.5, location=(1.0, 0, 2.2))
arm_r = bpy.context.active_object
arm_r.name = "Robot_ArmR"
arm_r.data.materials.append(mat_arm)
arm_r.parent = robot_root

# Gun barrel (on right arm)
bpy.ops.mesh.primitive_cylinder_add(radius=0.06, depth=0.8, location=(1.0, -0.0, 1.7))
gun = bpy.context.active_object
gun.name = "Robot_Gun"
gun.rotation_euler = (math.pi/2, 0, 0)  # Point forward
gun.data.materials.append(mat_gun)
gun.parent = robot_root

# Gun muzzle (glowing tip)
bpy.ops.mesh.primitive_uv_sphere_add(radius=0.08, location=(1.0, -0.4, 1.7))
muzzle = bpy.context.active_object
muzzle.name = "Robot_Muzzle"
muzzle.data.materials.append(mat_muzzle)
muzzle.parent = robot_root

# --- Improve Lighting ---
# Remove default light, add dramatic lighting
if "Light" in bpy.data.objects:
    bpy.data.objects.remove(bpy.data.objects["Light"], do_unlink=True)

# Key light (sun)
bpy.ops.object.light_add(type='SUN', location=(10, -10, 20))
sun = bpy.context.active_object
sun.name = "Sun_Key"
sun.data.energy = 3.0
sun.data.color = (1.0, 0.95, 0.85)
```

```
sun.rotation_euler = (math.radians(45), math.radians(10), math.radians(-30))

# Fill light
bpy.ops.object.light_add(type='AREA', location=(-8, 8, 10))
fill = bpy.context.active_object
fill.name = "Fill_Light"
fill.data.energy = 200
fill.data.color = (0.7, 0.8, 1.0)
fill.data.size = 5

# --- Set up camera ---
cam = bpy.data.objects["Camera"]
cam.location = (12, -18, 8)
cam.rotation_euler = (math.radians(72), 0, math.radians(30))

print("Scene setup complete! Robot built with colorful parts.")

```

2 Code Block #2 3786 bytes ✓ Executed

```

import bpy
import math
from mathutils import Vector

# --- Animate Spaceships ---
# 240 frames, 24fps, 10 seconds

# Ship 1: Kezrek G1 Spaceship (standalone mesh) - circular orbit
ship1 = bpy.data.objects["Kezrek G1 Spaceship"]
radius1 = 15
height1_base = 14
for frame in range(1, 241, 4):  # keyframe every 4 frames for smooth motion
    t = (frame - 1) / 239.0  # 0 to 1
    angle = t * math.pi * 3  # 1.5 full orbits
    x = radius1 * math.cos(angle)
    y = radius1 * math.sin(angle)
    z = height1_base + 3 * math.sin(angle * 2)  # bobbing up and down

    ship1.location = (x, y, z)
    ship1.keyframe_insert(data_path="location", frame=frame)

    # Face direction of travel (tangent)
    next_angle = angle + 0.05
    dx = -radius1 * math.sin(next_angle)
    dy = radius1 * math.cos(next_angle)
```

```
    yaw = math.atan2(dy, dx)
    # Add some banking into turns
    bank = math.sin(angle * 2) * 0.3
    ship1.rotation_euler = (bank, 0, yaw)
    ship1.keyframe_insert(data_path="rotation_euler", frame=frame)

# Ship 2: Sci Fi Spacecraft (empty parent) - figure-8 path
ship2 = bpy.data.objects["Sci Fi Spacecraft"]
for frame in range(1, 241, 4):
    t = (frame - 1) / 239.0
    angle = t * math.pi * 4  # 2 full figure-8 loops
    x = 14 * math.sin(angle)
    y = 10 * math.sin(angle * 2)  # figure-8 in Y
    z = 16 + 4 * math.cos(angle * 1.5)  # varying height

    ship2.location = (x, y, z)
    ship2.keyframe_insert(data_path="location", frame=frame)

    # Face direction of travel
    next_t = t + 0.01
    next_angle = next_t * math.pi * 4
    nx = 14 * math.sin(next_angle)
    ny = 10 * math.sin(next_angle * 2)
    dx = nx - x
    dy = ny - y
    yaw = math.atan2(dy, dx)
    bank = math.sin(angle) * 0.4
    ship2.rotation_euler = (bank * 0.5, -bank * 0.3, yaw)
    ship2.keyframe_insert(data_path="rotation_euler", frame=frame)

# Ship 3: Phantom Starship (empty parent) - sweeping diagonal passes
ship3 = bpy.data.objects["Phantom Starship"]
for frame in range(1, 241, 4):
    t = (frame - 1) / 239.0
    angle = t * math.pi * 2.5  # complex orbit
    x = 18 * math.cos(angle) + 5 * math.sin(angle * 3)
    y = 12 * math.sin(angle) + 4 * math.cos(angle * 2)
    z = 10 + 6 * math.sin(angle * 1.3) + 2 * math.cos(angle * 3)

    ship3.location = (x, y, z)
    ship3.keyframe_insert(data_path="location", frame=frame)

    # Face direction of travel
    next_t = t + 0.01
    na = next_t * math.pi * 2.5
    nx = 18 * math.cos(na) + 5 * math.sin(na * 3)
    ny = 12 * math.sin(na) + 4 * math.cos(na * 2)
```

```
    nz = 10 + 6 * math.sin(na * 1.3) + 2 * math.cos(na * 3)
    dx, dy, dz = nx - x, ny - y, nz - z
    yaw = math.atan2(dy, dx)
    pitch = math.atan2(dz, math.sqrt(dx*dx + dy*dy))
    bank = math.sin(angle * 2) * 0.35
    ship3.rotation_euler = (-pitch, bank, yaw)
    ship3.keyframe_insert(data_path="rotation_euler", frame=frame)

# --- Animate Robot rotating to track ships ---
robot_root = bpy.data.objects["Robot_Root"]
robot_pos = Vector((0, 0, 2.0))  # approximate torso height

for frame in range(1, 241, 2):
    t = (frame - 1) / 239.0
    bpy.context.scene.frame_set(frame)

    # Find nearest ship
    ships = [ship1, ship2, ship3]
    min_dist = float('inf')
    target = None
    for s in ships:
        d = (s.matrix_world.translation - robot_pos).length
        if d < min_dist:
            min_dist = d
            target = s

    # Rotate robot to face nearest ship
    target_loc = target.matrix_world.translation
    direction = target_loc - robot_pos
    yaw = math.atan2(direction.x, -direction.y)  # robot faces -Y by default

    robot_root.rotation_euler = (0, 0, yaw)
    robot_root.keyframe_insert(data_path="rotation_euler", frame=frame)

print("Spaceship flight paths and robot tracking animated!")

```

3 Code Block #3 4193 bytes ✓ Executed

```

import bpy
import math
from mathutils import Vector

# --- Create bullet material ---
if "Bullet_Mat" not in bpy.data.materials:
```

```python
    mat_bullet = bpy.data.materials.new("Bullet_Mat")
    mat_bullet.use_nodes = True
    bsdf = mat_bullet.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Base Color"].default_value = (1.0, 0.3, 0.0, 1)
    bsdf.inputs["Emission Color"].default_value = (1.0, 0.4, 0.0, 1)
    bsdf.inputs["Emission Strength"].default_value = 15.0
    bsdf.inputs["Roughness"].default_value = 0.0
else:
    mat_bullet = bpy.data.materials["Bullet_Mat"]

# --- Create bullet trail material ---
if "BulletTrail_Mat" not in bpy.data.materials:
    mat_trail = bpy.data.materials.new("BulletTrail_Mat")
    mat_trail.use_nodes = True
    bsdf = mat_trail.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Base Color"].default_value = (1.0, 0.6, 0.1, 1)
    bsdf.inputs["Emission Color"].default_value = (1.0, 0.5, 0.0, 1)
    bsdf.inputs["Emission Strength"].default_value = 8.0
else:
    mat_trail = bpy.data.materials["BulletTrail_Mat"]

# Ships to target
ship1 = bpy.data.objects["Kezrek G1 Spaceship"]
ship2 = bpy.data.objects["Sci Fi Spacecraft"]
ship3 = bpy.data.objects["Phantom Starship"]
ships = [ship1, ship2, ship3]

robot_pos = Vector((0, 0, 2.0))

# Fire a bullet every 25 frames (~1 second), so about 9 bullets over 240 frames
fire_frames = list(range(10, 236, 25))
bullet_speed_frames = 18  # bullet travels for 18 frames

# Delete any old bullets
old_bullets = [o for o in bpy.data.objects if o.name.startswith("Bullet_")]
for ob in old_bullets:
    bpy.data.objects.remove(ob, do_unlink=True)

for i, fire_frame in enumerate(fire_frames):
    # Determine muzzle position at fire time
    bpy.context.scene.frame_set(fire_frame)
    muzzle = bpy.data.objects["Robot_Muzzle"]
    start_pos = muzzle.matrix_world.translation.copy()

    # Find nearest ship at fire time
    min_dist = float('inf')
    target = None
```

```
    for s in ships:
        d = (s.matrix_world.translation - robot_pos).length
        if d < min_dist:
            min_dist = d
            target = s

    # Get target position (slightly ahead of current position for lead)
    target_pos = target.matrix_world.translation.copy()

    # Create bullet sphere
    bpy.ops.mesh.primitive_uv_sphere_add(radius=0.12, location=start_pos, segments=8, r
    bullet = bpy.context.active_object
    bullet.name = f"Bullet_{i:02d}"
    bullet.data.materials.append(mat_bullet)

    # Calculate direction
    direction = (target_pos - start_pos)
    travel_dist = direction.length
    direction.normalize()

    # Animate bullet: hidden before fire, visible during flight, hidden after
    # Frame before fire: hide bullet far away and scale to 0
    bullet.location = start_pos
    bullet.scale = (0, 0, 0)
    bullet.keyframe_insert(data_path="location", frame=fire_frame - 1)
    bullet.keyframe_insert(data_path="scale", frame=fire_frame - 1)

    # Frame of fire: appear at muzzle
    bullet.scale = (1, 1, 1)
    bullet.keyframe_insert(data_path="location", frame=fire_frame)
    bullet.keyframe_insert(data_path="scale", frame=fire_frame)

    # Bullet flies toward target over bullet_speed_frames
    end_frame = min(fire_frame + bullet_speed_frames, 240)

    # Interpolate bullet position (straight line, overshooting slightly past target)
    total_travel = travel_dist * 1.3  # overshoot
    for f in range(fire_frame, end_frame + 1, 3):
        progress = (f - fire_frame) / bullet_speed_frames
        pos = start_pos + direction * total_travel * progress
        bullet.location = pos
        bullet.keyframe_insert(data_path="location", frame=f)

    # After arriving: shrink and hide
    end_pos = start_pos + direction * total_travel
    bullet.location = end_pos
    bullet.scale = (1, 1, 1)
```

```
    bullet.keyframe_insert(data_path="location", frame=end_frame)
    bullet.keyframe_insert(data_path="scale", frame=end_frame)

    bullet.scale = (0, 0, 0)
    bullet.keyframe_insert(data_path="scale", frame=end_frame + 1)

# Reset to frame 1
bpy.context.scene.frame_set(1)

print(f"Created {len(fire_frames)} bullets firing at spaceships!")

```

4 Code Block #4 2748 bytes ✓ Executed

```

import bpy
import math

# --- Muzzle flash: animate the Robot_Muzzle emission to flash when firing ---
muzzle = bpy.data.objects["Robot_Muzzle"]
mat = muzzle.data.materials[0]
bsdf = mat.node_tree.nodes["Principled BSDF"]
emission_input = bsdf.inputs["Emission Strength"]

# Set base emission low
fire_frames = list(range(10, 236, 25))

# Keyframe emission strength: dim normally, bright flash on fire
for frame in range(1, 241, 4):
    emission_input.default_value = 1.0
    emission_input.keyframe_insert("default_value", frame=frame)

for fire_frame in fire_frames:
    # Flash on
    emission_input.default_value = 50.0
    emission_input.keyframe_insert("default_value", frame=fire_frame)
    # Flash off quickly
    emission_input.default_value = 1.0
    emission_input.keyframe_insert("default_value", frame=fire_frame + 3)

# --- Antenna glow pulsing ---
antenna = bpy.data.objects["Robot_AntennaBall"]
ant_mat = antenna.data.materials[0]
ant_bsdf = ant_mat.node_tree.nodes["Principled BSDF"]
ant_emission = ant_bsdf.inputs["Emission Strength"]

```

```python
for frame in range(1, 241, 6):
    t = (frame - 1) / 239.0
    pulse = 5 + 8 * abs(math.sin(t * math.pi * 12))
    ant_emission.default_value = pulse
    ant_emission.keyframe_insert("default_value", frame=frame)

# --- Adjust camera to frame the action well ---
cam = bpy.data.objects["Camera"]
# Animate camera slowly panning around the scene
for frame in range(1, 241, 8):
    t = (frame - 1) / 239.0
    angle = math.radians(-20) + t * math.radians(60)  # pan 60 degrees over animation
    dist = 28
    cam_x = dist * math.cos(angle)
    cam_y = -dist * math.sin(angle)
    cam_z = 10 + 3 * math.sin(t * math.pi)  # slight height variation

    cam.location = (cam_x, cam_y, cam_z)
    cam.keyframe_insert(data_path="location", frame=frame)

    # Point camera toward the action (slightly above origin where ships are)
    look_target = (0, 0, 8)
    dx = look_target[0] - cam_x
    dy = look_target[1] - cam_y
    dz = look_target[2] - cam_z

    dist_xy = math.sqrt(dx*dx + dy*dy)
    pitch = math.atan2(dist_xy, -dz)
    yaw = math.atan2(dx, -dy)

    cam.rotation_euler = (pitch, 0, yaw)
    cam.keyframe_insert(data_path="rotation_euler", frame=frame)

# --- Set world background to dark blue-black sky ---
world = bpy.context.scene.world
if world is None:
    world = bpy.data.worlds.new("World")
    bpy.context.scene.world = world
world.use_nodes = True
bg_node = world.node_tree.nodes.get("Background")
if bg_node:
    bg_node.inputs["Color"].default_value = (0.01, 0.01, 0.05, 1)  # dark night sky
    bg_node.inputs["Strength"].default_value = 0.3

bpy.context.scene.frame_set(1)
print("Muzzle flash, antenna pulse, camera animation, and sky setup complete!")
```

5 Code Block #5 2912 bytes ✓ Executed

```python

import bpy
import math
from mathutils import Vector

# --- STEP 1: Setup & Ship Flight Paths ---

# Extend timeline to 20 seconds (480 frames at 24fps)
scene = bpy.context.scene
scene.frame_start = 1
scene.frame_end = 480
scene.frame_current = 1

# --- Helper: clear all animation data from an object ---
def clear_anim(obj):
    if obj.animation_data:
        obj.animation_data_clear()

# --- Helper: duplicate a hierarchy (parent + all descendants) ---
def duplicate_hierarchy(parent_name, suffix):
    """Duplicate an object and all its children, return the new parent."""
    parent = bpy.data.objects[parent_name]

    # Deselect all
    bpy.ops.object.select_all(action='DESELECT')

    # Select parent and all descendants
    def select_recursive(obj):
        obj.select_set(True)
        for child in obj.children:
            select_recursive(child)

    select_recursive(parent)
    bpy.context.view_layer.objects.active = parent

    # Duplicate
    bpy.ops.object.duplicate()

    # The new parent is the active object
    new_parent = bpy.context.active_object
    new_parent.name = f"{parent_name}{suffix}"

    bpy.ops.object.select_all(action='DESELECT')
```

```python
    return new_parent

# --- Clear existing animation on all ships and robot ---
for name in ["Kezrek G1 Spaceship", "Sci Fi Spacecraft", "Phantom Starship",
             "Robot_Root", "Camera"]:
    if name in bpy.data.objects:
        clear_anim(bpy.data.objects[name])

# Clear old bullet objects
old_bullets = [o for o in bpy.data.objects if o.name.startswith("Bullet_")]
for ob in old_bullets:
    bpy.data.objects.remove(ob, do_unlink=True)

# Clear muzzle and antenna animation
for name in ["Robot_Muzzle", "Robot_AntennaBall"]:
    obj = bpy.data.objects[name]
    mat = obj.data.materials[0]
    bsdf = mat.node_tree.nodes["Principled BSDF"]
    inp = bsdf.inputs["Emission Strength"]
    # Clear keyframes on the material
    if mat.node_tree.animation_data:
        mat.node_tree.animation_data_clear()

# --- Duplicate spaceships ---
# Kezrek: make 2 copies (3 total)
kezrek2 = bpy.data.objects["Kezrek G1 Spaceship"].copy()
kezrek2.data = bpy.data.objects["Kezrek G1 Spaceship"].data.copy()
kezrek2.name = "Kezrek_Copy2"
scene.collection.objects.link(kezrek2)

kezrek3 = bpy.data.objects["Kezrek G1 Spaceship"].copy()
kezrek3.data = bpy.data.objects["Kezrek G1 Spaceship"].data.copy()
kezrek3.name = "Kezrek_Copy3"
scene.collection.objects.link(kezrek3)

# Sci Fi Spacecraft: make 1 copy (2 total)
scifi2 = duplicate_hierarchy("Sci Fi Spacecraft", "_Copy2")

# Phantom Starship: make 1 copy (2 total)
phantom2 = duplicate_hierarchy("Phantom Starship", "_Copy2")

print("Setup complete! Ships duplicated.")
print(f"  Kezrek ships: Kezrek G1 Spaceship, Kezrek_Copy2, Kezrek_Copy3")
print(f"  Sci Fi ships: Sci Fi Spacecraft, {scifi2.name}")
print(f"  Phantom ships: Phantom Starship, {phantom2.name}")

```

6 Code Block #6 10690 bytes ✓ Executed

```python

import bpy
import math
from mathutils import Vector

# --- Animate all 7 ships with unique flight paths ---
# All fly higher (18-35 range), 480 frames total

# Store ship info for bullet targeting later
# Two ships will crash: Kezrek_Copy3 (hit ~frame 280) and Phantom Starship (hit ~frame

CRASH_SHIP_1 = "Kezrek_Copy3"      # crashes at frame 285
CRASH_SHIP_2 = "Phantom Starship"  # crashes at frame 405
CRASH_1_FRAME = 285
CRASH_2_FRAME = 405
CRASH_DURATION = 80  # frames to spiral down

def animate_flight(obj, path_func, frame_start=1, frame_end=480, step=3):
    """Animate an object along a parametric path."""
    for frame in range(frame_start, frame_end + 1, step):
        t = (frame - frame_start) / max(frame_end - frame_start, 1)
        pos, rot = path_func(t, frame)
        obj.location = pos
        obj.keyframe_insert(data_path="location", frame=frame)
        obj.rotation_euler = rot
        obj.keyframe_insert(data_path="rotation_euler", frame=frame)

def facing_direction(x, y, z, nx, ny, nz, bank=0.0):
    """Calculate rotation to face direction of travel."""
    dx, dy, dz = nx - x, ny - y, nz - z
    yaw = math.atan2(dy, dx)
    dist_xy = math.sqrt(dx*dx + dy*dy)
    pitch = math.atan2(dz, dist_xy) if dist_xy > 0.01 else 0
    return (-pitch, bank, yaw)

# ============================================================
# SHIP 1: Kezrek G1 Spaceship — wide clockwise circle, high
# ============================================================
ship1 = bpy.data.objects["Kezrek G1 Spaceship"]
def path_kezrek1(t, frame):
    angle = -t * math.pi * 5  # 2.5 orbits, clockwise
    r = 22
    x = r * math.cos(angle)
    y = r * math.sin(angle)
```

```
    z = 22 + 4 * math.sin(angle * 2.5)
    # Next point for facing
    dt = 0.002
    na = -(t + dt) * math.pi * 5
    nx, ny = r * math.cos(na), r * math.sin(na)
    nz = 22 + 4 * math.sin(na * 2.5)
    bank = math.sin(angle * 2) * 0.25
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship1, path_kezrek1)

# ============================================================
# SHIP 2: Kezrek_Copy2 — tight counter-clockwise circle, higher
# ============================================================
ship2 = bpy.data.objects["Kezrek_Copy2"]
def path_kezrek2(t, frame):
    angle = t * math.pi * 6  # 3 orbits, counter-clockwise
    r = 16
    x = r * math.cos(angle) + 3
    y = r * math.sin(angle) - 2
    z = 26 + 3 * math.cos(angle * 3)
    dt = 0.002
    na = (t + dt) * math.pi * 6
    nx = r * math.cos(na) + 3
    ny = r * math.sin(na) - 2
    nz = 26 + 3 * math.cos(na * 3)
    bank = -math.sin(angle) * 0.3
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship2, path_kezrek2)

# ============================================================
# SHIP 3: Kezrek_Copy3 — elliptical orbit, CRASHES at frame 285
# ============================================================
ship3 = bpy.data.objects["Kezrek_Copy3"]
def path_kezrek3_normal(t, frame):
    angle = t * math.pi * 4
    x = 20 * math.cos(angle)
    y = 14 * math.sin(angle)
    z = 20 + 5 * math.sin(angle * 1.8)
    dt = 0.002
    na = (t + dt) * math.pi * 4
    nx = 20 * math.cos(na)
    ny = 14 * math.sin(na)
    nz = 20 + 5 * math.sin(na * 1.8)
```

```
    bank = math.sin(angle) * 0.3
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

# Normal flight up to crash frame
animate_flight(ship3, path_kezrek3_normal, frame_start=1, frame_end=CRASH_1_FRAME, step

# Get position/rotation at crash moment
crash1_t = (CRASH_1_FRAME - 1) / 479.0
crash1_pos, crash1_rot = path_kezrek3_normal(crash1_t, CRASH_1_FRAME)

# Crash: spiral down while spinning
for frame in range(CRASH_1_FRAME, min(CRASH_1_FRAME + CRASH_DURATION, 481), 2):
    progress = (frame - CRASH_1_FRAME) / CRASH_DURATION
    # Ease-in fall (accelerating)
    fall = progress * progress * crash1_pos[2]

    # Drift forward slightly while falling
    drift_angle = crash1_t * math.pi * 4
    drift_x = crash1_pos[0] + math.cos(drift_angle) * progress * 8
    drift_y = crash1_pos[1] + math.sin(drift_angle) * progress * 8

    x = drift_x + math.sin(progress * 15) * (1 - progress) * 3  # wobble
    y = drift_y + math.cos(progress * 15) * (1 - progress) * 3
    z = max(crash1_pos[2] - fall, 0.5)  # don't go below ground

    # Spinning wildly
    spin = progress * math.pi * 12
    tumble = progress * math.pi * 8

    ship3.location = (x, y, z)
    ship3.keyframe_insert(data_path="location", frame=frame)
    ship3.rotation_euler = (tumble, spin * 0.7, crash1_rot[2] + spin)
    ship3.keyframe_insert(data_path="rotation_euler", frame=frame)

# Stay on ground after crash
final_frame = min(CRASH_1_FRAME + CRASH_DURATION, 480)
ship3.keyframe_insert(data_path="location", frame=final_frame + 1)
ship3.keyframe_insert(data_path="rotation_euler", frame=final_frame + 1)

# ============================================================
# SHIP 4: Sci Fi Spacecraft — large figure-8
# ============================================================
ship4 = bpy.data.objects["Sci Fi Spacecraft"]
def path_scifi1(t, frame):
    angle = t * math.pi * 5
    x = 18 * math.sin(angle)
```

```
    y = 12 * math.sin(angle * 2)
    z = 24 + 5 * math.cos(angle * 1.3)
    dt = 0.002
    na = (t + dt) * math.pi * 5
    nx = 18 * math.sin(na)
    ny = 12 * math.sin(na * 2)
    nz = 24 + 5 * math.cos(na * 1.3)
    bank = math.cos(angle) * 0.35
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship4, path_scifi1)

# ============================================================
# SHIP 5: Sci Fi Spacecraft_Copy2 — evasive zigzag with altitude changes
# ============================================================
ship5 = bpy.data.objects["Sci Fi Spacecraft_Copy2"]
def path_scifi2(t, frame):
    angle = t * math.pi * 4
    # Zigzag: sine wave superimposed on circle
    base_x = 15 * math.cos(angle)
    base_y = 15 * math.sin(angle)
    # Evasive maneuvers: sharp direction changes
    evade_x = 6 * math.sin(angle * 7) * math.cos(angle * 3)
    evade_y = 6 * math.cos(angle * 5) * math.sin(angle * 4)
    x = base_x + evade_x
    y = base_y + evade_y
    z = 28 + 4 * math.sin(angle * 6) + 3 * math.cos(angle * 2)
    dt = 0.002
    na = (t + dt) * math.pi * 4
    nx = 15 * math.cos(na) + 6 * math.sin(na * 7) * math.cos(na * 3)
    ny = 15 * math.sin(na) + 6 * math.cos(na * 5) * math.sin(na * 4)
    nz = 28 + 4 * math.sin(na * 6) + 3 * math.cos(na * 2)
    bank = math.sin(angle * 5) * 0.4
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship5, path_scifi2)

# ============================================================
# SHIP 6: Phantom Starship — complex swooping, CRASHES at frame 405
# ============================================================
ship6 = bpy.data.objects["Phantom Starship"]
def path_phantom1_normal(t, frame):
    angle = t * math.pi * 3.5
    x = 20 * math.cos(angle) + 7 * math.sin(angle * 2.5)
    y = 16 * math.sin(angle) + 5 * math.cos(angle * 1.8)
```

```
    z = 18 + 8 * math.sin(angle * 1.2) + 3 * math.cos(angle * 3.5)
    dt = 0.002
    na = (t + dt) * math.pi * 3.5
    nx = 20 * math.cos(na) + 7 * math.sin(na * 2.5)
    ny = 16 * math.sin(na) + 5 * math.cos(na * 1.8)
    nz = 18 + 8 * math.sin(na * 1.2) + 3 * math.cos(na * 3.5)
    bank = math.sin(angle * 2) * 0.35
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship6, path_phantom1_normal, frame_start=1, frame_end=CRASH_2_FRAME, ste

# Crash sequence for Phantom
crash2_t = (CRASH_2_FRAME - 1) / 479.0
crash2_pos, crash2_rot = path_phantom1_normal(crash2_t, CRASH_2_FRAME)

for frame in range(CRASH_2_FRAME, min(CRASH_2_FRAME + CRASH_DURATION, 481), 2):
    progress = (frame - CRASH_2_FRAME) / CRASH_DURATION
    fall = progress * progress * crash2_pos[2]

    drift_angle = crash2_t * math.pi * 3.5
    drift_x = crash2_pos[0] + math.cos(drift_angle) * progress * 10
    drift_y = crash2_pos[1] + math.sin(drift_angle) * progress * 10

    x = drift_x + math.sin(progress * 18) * (1 - progress) * 4
    y = drift_y + math.cos(progress * 18) * (1 - progress) * 4
    z = max(crash2_pos[2] - fall, 0.5)

    spin = progress * math.pi * 14
    tumble = progress * math.pi * 6

    ship6.location = (x, y, z)
    ship6.keyframe_insert(data_path="location", frame=frame)
    ship6.rotation_euler = (tumble * 0.8, spin * 0.6, crash2_rot[2] + spin)
    ship6.keyframe_insert(data_path="rotation_euler", frame=frame)

final_frame2 = min(CRASH_2_FRAME + CRASH_DURATION, 480)
ship6.keyframe_insert(data_path="location", frame=final_frame2 + 1)
ship6.keyframe_insert(data_path="rotation_euler", frame=final_frame2 + 1)

# ============================================================
# SHIP 7: Phantom Starship_Copy2 — fast weaving orbit, high altitude
# ============================================================
ship7 = bpy.data.objects["Phantom Starship_Copy2"]
def path_phantom2(t, frame):
    angle = t * math.pi * 6
    x = 14 * math.cos(angle) + 8 * math.cos(angle * 0.3)
```

```
    y = 14 * math.sin(angle) + 8 * math.sin(angle * 0.7)
    z = 30 + 5 * math.sin(angle * 2) + 3 * math.cos(angle * 4)
    dt = 0.002
    na = (t + dt) * math.pi * 6
    nx = 14 * math.cos(na) + 8 * math.cos(na * 0.3)
    ny = 14 * math.sin(na) + 8 * math.sin(na * 0.7)
    nz = 30 + 5 * math.sin(na * 2) + 3 * math.cos(na * 4)
    bank = math.sin(angle * 3) * 0.3
    rot = facing_direction(x, y, z, nx, ny, nz, bank)
    return (x, y, z), rot

animate_flight(ship7, path_phantom2)

# --- Robot tracking: rotate to face nearest ship each frame ---
robot_root = bpy.data.objects["Robot_Root"]
# Clear old animation
if robot_root.animation_data:
    robot_root.animation_data_clear()

all_ships = [ship1, ship2, ship3, ship4, ship5, ship6, ship7]
robot_pos = Vector((0, 0, 2.0))

for frame in range(1, 481, 2):
    bpy.context.scene.frame_set(frame)

    min_dist = float('inf')
    target = None
    for s in all_ships:
        d = (s.matrix_world.translation - robot_pos).length
        if d < min_dist:
            min_dist = d
            target = s

    target_loc = target.matrix_world.translation
    direction = target_loc - robot_pos
    yaw = math.atan2(direction.x, -direction.y)

    robot_root.rotation_euler = (0, 0, yaw)
    robot_root.keyframe_insert(data_path="rotation_euler", frame=frame)

bpy.context.scene.frame_set(1)
print("All 7 ship flight paths animated! 2 crash sequences set up.")
print(f"  {CRASH_SHIP_1} crashes at frame {CRASH_1_FRAME}")
print(f"  {CRASH_SHIP_2} crashes at frame {CRASH_2_FRAME}")

```

7 Code Block #7 5516 bytes ✓ Executed

```python

import bpy
import math
from mathutils import Vector
import random

random.seed(42)  # Reproducible

# --- Bullet Stream System ---
# Each burst: 10 bullets fired 3 frames apart (burst spans 27 frames)
# Bursts happen at these frames:
BURSTS = [
    {"start": 30,  "target_ship": None,  "hit": False},  # miss - shoots at nearest
    {"start": 90,  "target_ship": None,  "hit": False},  # miss
    {"start": 155, "target_ship": None,  "hit": False},  # miss
    {"start": 220, "target_ship": None,  "hit": False},  # miss
    {"start": 270, "target_ship": "Kezrek_Copy3", "hit": True},   # HIT! → crash at ~28
    {"start": 310, "target_ship": None,  "hit": False},  # miss
    {"start": 370, "target_ship": None,  "hit": False},  # miss
    {"start": 390, "target_ship": "Phantom Starship", "hit": True},  # HIT! → crash at
    {"start": 430, "target_ship": None,  "hit": False},  # miss
    {"start": 460, "target_ship": None,  "hit": False},  # miss (victory shots)
]

BULLETS_PER_BURST = 10
BULLET_INTERVAL = 3  # frames between bullets in a burst
BULLET_FLIGHT = 15   # frames for bullet to reach target area

# Bullet material
if "Bullet_Mat" not in bpy.data.materials:
    mat_bullet = bpy.data.materials.new("Bullet_Mat")
    mat_bullet.use_nodes = True
    bsdf = mat_bullet.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Base Color"].default_value = (1.0, 0.3, 0.0, 1)
    bsdf.inputs["Emission Color"].default_value = (1.0, 0.4, 0.0, 1)
    bsdf.inputs["Emission Strength"].default_value = 15.0
else:
    mat_bullet = bpy.data.materials["Bullet_Mat"]

# Create a shared bullet mesh (small sphere)
if "BulletMesh" not in bpy.data.meshes:
    bpy.ops.mesh.primitive_uv_sphere_add(radius=0.1, segments=8, ring_count=6, location
    temp = bpy.context.active_object
    bullet_mesh = temp.data
```

```
    bullet_mesh.name = "BulletMesh"
    bpy.data.objects.remove(temp, do_unlink=True)
else:
    bullet_mesh = bpy.data.meshes["BulletMesh"]

# All ships for targeting
all_ships = [
    bpy.data.objects["Kezrek G1 Spaceship"],
    bpy.data.objects["Kezrek_Copy2"],
    bpy.data.objects["Kezrek_Copy3"],
    bpy.data.objects["Sci Fi Spacecraft"],
    bpy.data.objects["Sci Fi Spacecraft_Copy2"],
    bpy.data.objects["Phantom Starship"],
    bpy.data.objects["Phantom Starship_Copy2"],
]

robot_pos = Vector((0, 0, 2.0))
muzzle_obj = bpy.data.objects["Robot_Muzzle"]

bullet_index = 0

for burst in BURSTS:
    burst_start = burst["start"]
    is_hit = burst["hit"]
    target_name = burst["target_ship"]

    for b in range(BULLETS_PER_BURST):
        fire_frame = burst_start + b * BULLET_INTERVAL
        if fire_frame > 475:
            break

        bpy.context.scene.frame_set(fire_frame)

        # Get muzzle position
        start_pos = muzzle_obj.matrix_world.translation.copy()

        # Determine target
        if target_name and target_name in bpy.data.objects:
            target = bpy.data.objects[target_name]
        else:
            # Find nearest ship
            min_dist = float('inf')
            target = all_ships[0]
            for s in all_ships:
                d = (s.matrix_world.translation - robot_pos).length
                if d < min_dist:
                    min_dist = d
```

```
                    target = s

        target_pos = target.matrix_world.translation.copy()

        # Add spread/inaccuracy
        if is_hit and b ≥ 6:
            # Last few bullets in a hit burst are accurate
            spread = Vector((random.uniform(-0.5, 0.5), random.uniform(-0.5, 0.5), rand
        else:
            # Most bullets miss - add significant spread
            spread = Vector((random.uniform(-4, 4), random.uniform(-4, 4), random.unifo

        aim_pos = target_pos + spread

        # Create bullet object (linked duplicate for performance)
        bullet = bpy.data.objects.new(f"Bullet_{bullet_index:03d}", bullet_mesh)
        bpy.context.scene.collection.objects.link(bullet)
        if not bullet.data.materials:
            bullet.data.materials.append(mat_bullet)

        direction = (aim_pos - start_pos)
        total_dist = direction.length * 1.4  # overshoot
        direction.normalize()

        end_frame = min(fire_frame + BULLET_FLIGHT, 480)

        # Before fire: invisible (scale 0, at start position)
        bullet.location = start_pos
        bullet.scale = (0, 0, 0)
        bullet.keyframe_insert(data_path="location", frame=fire_frame - 1)
        bullet.keyframe_insert(data_path="scale", frame=fire_frame - 1)

        # Appear at fire
        bullet.scale = (1, 1, 1)
        bullet.keyframe_insert(data_path="location", frame=fire_frame)
        bullet.keyframe_insert(data_path="scale", frame=fire_frame)

        # Fly toward target
        for f in range(fire_frame + 3, end_frame + 1, 3):
            progress = (f - fire_frame) / BULLET_FLIGHT
            pos = start_pos + direction * total_dist * progress
            bullet.location = pos
            bullet.keyframe_insert(data_path="location", frame=f)

        # Disappear after flight
        bullet.scale = (0, 0, 0)
        bullet.keyframe_insert(data_path="scale", frame=end_frame + 1)
```

```python

        bullet_index += 1

bpy.context.scene.frame_set(1)
print(f"Created {bullet_index} bullets across {len(BURSTS)} bursts!")
print(f"  Hit bursts target: Kezrek_Copy3 (frame 270) and Phantom Starship (frame 390)"

```

8 Code Block #8 3654 bytes ✓ Executed

```python

import bpy
import math

# --- Muzzle flash: emission spikes during each burst ---
muzzle = bpy.data.objects["Robot_Muzzle"]
mat = muzzle.data.materials[0]
bsdf = mat.node_tree.nodes["Principled BSDF"]
emission_input = bsdf.inputs["Emission Strength"]

# Clear any old keyframes
if mat.node_tree.animation_data:
    mat.node_tree.animation_data_clear()

BURSTS_START = [30, 90, 155, 220, 270, 310, 370, 390, 430, 460]
BULLETS_PER_BURST = 10
BULLET_INTERVAL = 3

# Set base dim emission throughout
for frame in range(1, 481, 8):
    emission_input.default_value = 1.0
    emission_input.keyframe_insert("default_value", frame=frame)

# Flash during each bullet fire
for burst_start in BURSTS_START:
    for b in range(BULLETS_PER_BURST):
        fire_frame = burst_start + b * BULLET_INTERVAL
        if fire_frame > 475:
            break
        # Bright flash
        emission_input.default_value = 60.0
        emission_input.keyframe_insert("default_value", frame=fire_frame)
        # Quick decay
        emission_input.default_value = 2.0
        emission_input.keyframe_insert("default_value", frame=fire_frame + 2)

```

```python
# --- Antenna glow pulsing (faster during firing) ---
antenna = bpy.data.objects["Robot_AntennaBall"]
ant_mat = antenna.data.materials[0]
ant_bsdf = ant_mat.node_tree.nodes["Principled BSDF"]
ant_emission = ant_bsdf.inputs["Emission Strength"]

if ant_mat.node_tree.animation_data:
    ant_mat.node_tree.animation_data_clear()

for frame in range(1, 481, 4):
    t = (frame - 1) / 479.0
    # Check if we're in a burst
    in_burst = False
    for bs in BURSTS_START:
        if bs - 5 ≤ frame ≤ bs + BULLETS_PER_BURST * BULLET_INTERVAL + 5:
            in_burst = True
            break

    if in_burst:
        pulse = 8 + 12 * abs(math.sin(frame * 0.8))  # fast bright pulse
    else:
        pulse = 4 + 5 * abs(math.sin(t * math.pi * 15))  # calm pulse

    ant_emission.default_value = pulse
    ant_emission.keyframe_insert("default_value", frame=frame)

# --- Camera: wider, more dynamic for 20 seconds ---
cam = bpy.data.objects["Camera"]
if cam.animation_data:
    cam.animation_data_clear()

for frame in range(1, 481, 6):
    t = (frame - 1) / 479.0

    # Camera orbits wider, with dramatic moments
    base_angle = math.radians(-30) + t * math.radians(120)  # 120 degree pan

    # Add dramatic push-in during crash moments
    dist = 32
    if 270 ≤ frame ≤ 365:
        # Closer during first crash
        dist = 32 - 8 * math.sin((frame - 270) / 95 * math.pi)
    elif 390 ≤ frame ≤ 480:
        # Closer during second crash
        dist = 32 - 10 * math.sin((frame - 390) / 90 * math.pi)

    cam_x = dist * math.cos(base_angle)
```

```
    cam_y = -dist * math.sin(base_angle)
    cam_z = 12 + 5 * math.sin(t * math.pi * 1.5)

    # During crashes, tilt up to see falling ships
    look_z = 10
    if 285 ≤ frame ≤ 365:
        progress = (frame - 285) / 80
        look_z = 10 - progress * 6  # look down to follow crash
    elif 405 ≤ frame ≤ 480:
        progress = (frame - 405) / 75
        look_z = 10 - progress * 6

    cam.location = (cam_x, cam_y, cam_z)
    cam.keyframe_insert(data_path="location", frame=frame)

    look_target = (0, 0, look_z)
    dx = look_target[0] - cam_x
    dy = look_target[1] - cam_y
    dz = look_target[2] - cam_z
    dist_xy = math.sqrt(dx*dx + dy*dy)
    pitch = math.atan2(dist_xy, -dz)
    yaw = math.atan2(dx, -dy)

    cam.rotation_euler = (pitch, 0, yaw)
    cam.keyframe_insert(data_path="rotation_euler", frame=frame)

bpy.context.scene.frame_set(1)
print("Muzzle flash, antenna pulse, and cinematic camera all animated!")

```

9 Code Block #9 5808 bytes

```

import bpy
import math
import random
import bmesh
from mathutils import Vector

random.seed(99)

# ============================================================
# 1. BIGGER BULLETS — scale up all bullet objects 3x
# ============================================================
bullet_count = 0
for obj in bpy.data.objects:
```

```
    if obj.name.startswith("Bullet_"):
        # Scale up the keyframed scale values
        # We need to adjust the keyframes: where scale was (1,1,1) make it (3,3,3)
        # and where it was (0,0,0) keep it (0,0,0)
        if obj.animation_data and obj.animation_data.action:
            action = obj.animation_data.action
            # Can't use fcurves in Blender 5.x — just re-key the scale
        bullet_count += 1

# Since we can't access fcurves, let's re-keyframe bullet scales
# We know the pattern: scale 0 one frame before fire, scale 1 at fire, scale 0 after fl
# Instead of re-keying everything, just scale the mesh itself to be bigger
bullet_mesh = bpy.data.meshes.get("BulletMesh")
if bullet_mesh:
    bm = bmesh.new()
    bm.from_mesh(bullet_mesh)
    # Scale all verts by 3x (radius goes from 0.1 to 0.3)
    for v in bm.verts:
        v.co *= 3.0
    bm.to_mesh(bullet_mesh)
    bm.free()
    bullet_mesh.update()

# Also make the bullet material more visible — brighter emission
mat_bullet = bpy.data.materials.get("Bullet_Mat")
if mat_bullet:
    bsdf = mat_bullet.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Emission Strength"].default_value = 30.0
    # Make them more yellow-hot for visibility
    bsdf.inputs["Base Color"].default_value = (1.0, 0.6, 0.0, 1)
    bsdf.inputs["Emission Color"].default_value = (1.0, 0.7, 0.0, 1)

print(f"Scaled up {bullet_count} bullets (mesh 3x bigger, emission 30)")

# ============================================================
# 2. BIGGER TERRAIN — scale ground from 100 to 300
# ============================================================
ground = bpy.data.objects.get("Ground")
if ground:
    ground.scale = (3, 3, 1)  # 100 * 3 = 300 units across
    print("Ground scaled to 300x300")

# ============================================================
# 3. DARK SKY WITH TWINKLING STARS
# ============================================================
world = bpy.context.scene.world
if world is None:
```

```
    world = bpy.data.worlds.new("World")
    bpy.context.scene.world = world

world.use_nodes = True
nodes = world.node_tree.nodes
links = world.node_tree.links

# Clear existing nodes
for node in nodes:
    nodes.remove(node)

# Output node
output = nodes.new('ShaderNodeOutputWorld')
output.location = (600, 0)

# Mix shader: dark background + stars
mix = nodes.new('ShaderNodeMixShader')
mix.location = (400, 0)
links.new(mix.outputs[0], output.inputs['Surface'])

# Dark sky background
bg_dark = nodes.new('ShaderNodeBackground')
bg_dark.location = (0, 100)
bg_dark.inputs['Color'].default_value = (0.005, 0.005, 0.025, 1)  # very dark blue-blac
bg_dark.inputs['Strength'].default_value = 1.0
links.new(bg_dark.outputs[0], mix.inputs[1])

# Star emission background
bg_stars = nodes.new('ShaderNodeBackground')
bg_stars.location = (0, -100)
bg_stars.inputs['Color'].default_value = (1.0, 1.0, 0.95, 1)  # warm white stars
bg_stars.inputs['Strength'].default_value = 5.0
links.new(bg_stars.outputs[0], mix.inputs[2])

# Star pattern using noise texture → color ramp (threshold to get dots)
tex_coord = nodes.new('ShaderNodeTexCoord')
tex_coord.location = (-800, 0)

# Noise texture for star positions
noise = nodes.new('ShaderNodeTexNoise')
noise.location = (-600, 0)
noise.inputs['Scale'].default_value = 800.0      # lots of tiny dots
noise.inputs['Detail'].default_value = 0.0        # sharp noise
noise.inputs['Roughness'].default_value = 0.0
links.new(tex_coord.outputs['Object'], noise.inputs['Vector'])

# Color ramp to threshold: only the very brightest noise becomes stars
```

```python
ramp = nodes.new('ShaderNodeValToRGB')
ramp.location = (-300, 0)
ramp.color_ramp.interpolation = 'CONSTANT'
# Most of the range is black, only very top is white = sparse stars
ramp.color_ramp.elements[0].position = 0.0
ramp.color_ramp.elements[0].color = (0, 0, 0, 1)
ramp.color_ramp.elements[1].position = 0.82  # only top 18% are stars
ramp.color_ramp.elements[1].color = (1, 1, 1, 1)
links.new(noise.outputs['Fac'], ramp.inputs['Fac'])

# Second noise for star brightness variation
noise2 = nodes.new('ShaderNodeTexNoise')
noise2.location = (-600, -200)
noise2.inputs['Scale'].default_value = 400.0
noise2.inputs['Detail'].default_value = 2.0
links.new(tex_coord.outputs['Object'], noise2.inputs['Vector'])

# Multiply star mask × brightness variation for varied star brightness
multiply = nodes.new('ShaderNodeMath')
multiply.location = (-100, 0)
multiply.operation = 'MULTIPLY'
links.new(ramp.outputs['Color'], multiply.inputs[0])
links.new(noise2.outputs['Fac'], multiply.inputs[1])

# Use as mix factor
links.new(multiply.outputs[0], mix.inputs['Fac'])

# --- Add twinkling via animated noise ---
# We'll animate the star brightness strength using keyframes on bg_stars strength
# Subtle overall twinkle by animating the noise W value
# Use a mapping node to add animated offset for twinkling effect
mapping = nodes.new('ShaderNodeMapping')
mapping.location = (-700, -50)
links.new(tex_coord.outputs['Object'], mapping.inputs['Vector'])
links.new(mapping.outputs['Vector'], noise.inputs['Vector'])

# Animate the mapping Z offset to create subtle twinkling
for frame in range(1, 481, 8):
    t = (frame - 1) / 479.0
    # Very slow drift creates subtle twinkle
    mapping.inputs['Location'].default_value[2] = t * 0.3
    mapping.inputs['Location'].keyframe_insert("default_value", index=2, frame=frame)

print("Dark sky with twinkling stars created!")
print("Done! Bigger bullets, bigger terrain, starry sky.")
```

10 Code Block #10 4449 bytes ✓ Executed

```python

import bpy
import math
import random
import bmesh
from mathutils import Vector

random.seed(99)

# ============================================================
# 1. BIGGER BULLETS — scale up the shared mesh 3x
# ============================================================
bullet_mesh = bpy.data.meshes.get("BulletMesh")
if bullet_mesh:
    bm = bmesh.new()
    bm.from_mesh(bullet_mesh)
    for v in bm.verts:
        v.co *= 3.0
    bm.to_mesh(bullet_mesh)
    bm.free()
    bullet_mesh.update()
    print("Bullet mesh scaled 3x (radius 0.1 → 0.3)")

# Brighter, more visible bullet material
mat_bullet = bpy.data.materials.get("Bullet_Mat")
if mat_bullet:
    bsdf = mat_bullet.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Emission Strength"].default_value = 30.0
    bsdf.inputs["Base Color"].default_value = (1.0, 0.6, 0.0, 1)
    bsdf.inputs["Emission Color"].default_value = (1.0, 0.7, 0.0, 1)
    print("Bullet material: brighter yellow-hot emission (30)")

# ============================================================
# 2. BIGGER TERRAIN — scale ground to 300x300
# ============================================================
ground = bpy.data.objects.get("Ground")
if ground:
    ground.scale = (3, 3, 1)
    print("Ground scaled to 300x300")

# ============================================================
# 3. DARK SKY WITH TWINKLING STARS
# ============================================================
world = bpy.context.scene.world
```

```
if world is None:
    world = bpy.data.worlds.new("World")
    bpy.context.scene.world = world

world.use_nodes = True
nodes = world.node_tree.nodes
links = world.node_tree.links

# Clear existing nodes
for node in nodes:
    nodes.remove(node)

# Output node
output = nodes.new('ShaderNodeOutputWorld')
output.location = (600, 0)

# Mix shader: dark background + stars
mix = nodes.new('ShaderNodeMixShader')
mix.location = (400, 0)
links.new(mix.outputs[0], output.inputs['Surface'])

# Dark sky background
bg_dark = nodes.new('ShaderNodeBackground')
bg_dark.location = (0, 100)
bg_dark.inputs['Color'].default_value = (0.005, 0.005, 0.025, 1)
bg_dark.inputs['Strength'].default_value = 1.0
links.new(bg_dark.outputs[0], mix.inputs[1])

# Star emission background
bg_stars = nodes.new('ShaderNodeBackground')
bg_stars.location = (0, -100)
bg_stars.inputs['Color'].default_value = (1.0, 1.0, 0.95, 1)
bg_stars.inputs['Strength'].default_value = 5.0
links.new(bg_stars.outputs[0], mix.inputs[2])

# Texture coordinate
tex_coord = nodes.new('ShaderNodeTexCoord')
tex_coord.location = (-800, 0)

# Mapping node for animated twinkling
mapping = nodes.new('ShaderNodeMapping')
mapping.location = (-650, 0)
links.new(tex_coord.outputs['Object'], mapping.inputs['Vector'])

# Noise texture for star positions — high scale = tiny dots
noise = nodes.new('ShaderNodeTexNoise')
noise.location = (-450, 0)
```

```
noise.inputs['Scale'].default_value = 800.0
noise.inputs['Detail'].default_value = 0.0
noise.inputs['Roughness'].default_value = 0.0
links.new(mapping.outputs['Vector'], noise.inputs['Vector'])

# Color ramp: threshold so only brightest peaks become stars
ramp = nodes.new('ShaderNodeValToRGB')
ramp.location = (-250, 0)
ramp.color_ramp.interpolation = 'CONSTANT'
ramp.color_ramp.elements[0].position = 0.0
ramp.color_ramp.elements[0].color = (0, 0, 0, 1)
ramp.color_ramp.elements[1].position = 0.83
ramp.color_ramp.elements[1].color = (1, 1, 1, 1)
links.new(noise.outputs['Fac'], ramp.inputs['Fac'])

# Second noise for brightness variation (different scale)
noise2 = nodes.new('ShaderNodeTexNoise')
noise2.location = (-450, -200)
noise2.inputs['Scale'].default_value = 350.0
noise2.inputs['Detail'].default_value = 2.0
links.new(mapping.outputs['Vector'], noise2.inputs['Vector'])

# Multiply star mask × brightness for varied star sizes
multiply = nodes.new('ShaderNodeMath')
multiply.location = (-80, 0)
multiply.operation = 'MULTIPLY'
links.new(ramp.outputs['Color'], multiply.inputs[0])
links.new(noise2.outputs['Fac'], multiply.inputs[1])

# Feed into mix factor
links.new(multiply.outputs[0], mix.inputs['Fac'])

# Animate mapping Z offset for subtle twinkling drift
for frame in range(1, 481, 8):
    t = (frame - 1) / 479.0
    mapping.inputs['Location'].default_value[2] = t * 0.3
    mapping.inputs['Location'].keyframe_insert("default_value", index=2, frame=frame)

print("Dark sky with twinkling stars created!")
print("All done — bigger bullets, bigger terrain, starry night sky!")

```

11 Code Block #11 3730 bytes ✓ Executed

```

import bpy
```

```python

world = bpy.context.scene.world
nodes = world.node_tree.nodes
links = world.node_tree.links

# Clear everything and rebuild
for node in nodes:
    nodes.remove(node)

# Output
output = nodes.new('ShaderNodeOutputWorld')
output.location = (800, 0)

# Mix: dark sky + star glow
mix = nodes.new('ShaderNodeMixShader')
mix.location = (600, 0)
links.new(mix.outputs[0], output.inputs['Surface'])

# Dark sky
bg_dark = nodes.new('ShaderNodeBackground')
bg_dark.location = (200, 100)
bg_dark.inputs['Color'].default_value = (0.005, 0.005, 0.025, 1)
bg_dark.inputs['Strength'].default_value = 1.0
links.new(bg_dark.outputs[0], mix.inputs[1])

# Star glow
bg_stars = nodes.new('ShaderNodeBackground')
bg_stars.location = (200, -100)
bg_stars.inputs['Color'].default_value = (1.0, 1.0, 0.95, 1)
bg_stars.inputs['Strength'].default_value = 5.0
links.new(bg_stars.outputs[0], mix.inputs[2])

# --- STATIC star positions (no animation on this path) ---
tex_coord = nodes.new('ShaderNodeTexCoord')
tex_coord.location = (-800, 0)

# Main noise for star positions — STATIC, never animated
noise_stars = nodes.new('ShaderNodeTexNoise')
noise_stars.location = (-500, 0)
noise_stars.inputs['Scale'].default_value = 800.0
noise_stars.inputs['Detail'].default_value = 0.0
noise_stars.inputs['Roughness'].default_value = 0.0
links.new(tex_coord.outputs['Object'], noise_stars.inputs['Vector'])

# Threshold to get sparse star dots
ramp = nodes.new('ShaderNodeValToRGB')
ramp.location = (-300, 0)
```

```
ramp.color_ramp.interpolation = 'CONSTANT'
ramp.color_ramp.elements[0].position = 0.0
ramp.color_ramp.elements[0].color = (0, 0, 0, 1)
ramp.color_ramp.elements[1].position = 0.83
ramp.color_ramp.elements[1].color = (1, 1, 1, 1)
links.new(noise_stars.outputs['Fac'], ramp.inputs['Fac'])

# --- TWINKLE: separate animated noise that modulates brightness ---
# This noise uses a different scale so it doesn't correlate with star positions
# We animate its W coordinate (4D noise) for smooth temporal variation
mapping_twinkle = nodes.new('ShaderNodeMapping')
mapping_twinkle.location = (-650, -250)
links.new(tex_coord.outputs['Object'], mapping_twinkle.inputs['Vector'])

noise_twinkle = nodes.new('ShaderNodeTexNoise')
noise_twinkle.location = (-450, -250)
noise_twinkle.inputs['Scale'].default_value = 150.0  # coarser — twinkle zones
noise_twinkle.inputs['Detail'].default_value = 3.0
noise_twinkle.inputs['Roughness'].default_value = 0.7
links.new(mapping_twinkle.outputs['Vector'], noise_twinkle.inputs['Vector'])

# Remap twinkle noise so stars dim between 0.4 and 1.0 (never fully disappear)
twinkle_ramp = nodes.new('ShaderNodeValToRGB')
twinkle_ramp.location = (-250, -250)
twinkle_ramp.color_ramp.elements[0].position = 0.0
twinkle_ramp.color_ramp.elements[0].color = (0.4, 0.4, 0.4, 1)  # dimmest
twinkle_ramp.color_ramp.elements[1].position = 1.0
twinkle_ramp.color_ramp.elements[1].color = (1, 1, 1, 1)  # brightest
links.new(noise_twinkle.outputs['Fac'], twinkle_ramp.inputs['Fac'])

# Multiply: star mask × twinkle brightness
multiply = nodes.new('ShaderNodeMath')
multiply.location = (-50, 0)
multiply.operation = 'MULTIPLY'
links.new(ramp.outputs['Color'], multiply.inputs[0])
links.new(twinkle_ramp.outputs['Color'], multiply.inputs[1])

links.new(multiply.outputs[0], mix.inputs['Fac'])

# Animate ONLY the twinkle mapping offset — slow gentle drift
# This shifts which stars are bright/dim over time, not which stars exist
for frame in range(1, 481, 6):
    t = (frame - 1) / 479.0
    mapping_twinkle.inputs['Location'].default_value[2] = t * 2.0
    mapping_twinkle.inputs['Location'].keyframe_insert("default_value", index=2, frame=f

print("Fixed! Stars stay visible throughout — twinkle brightness varies gently.")
```

```

```

12 Code Block #12 2196 bytes ✓ Executed

```

import bpy

world = bpy.context.scene.world
nodes = world.node_tree.nodes
links = world.node_tree.links

# Clear everything
for node in nodes:
    nodes.remove(node)

# Output
output = nodes.new('ShaderNodeOutputWorld')
output.location = (600, 0)

# --- Use Generated coordinates (stable for world shader = view direction) ---
tex_coord = nodes.new('ShaderNodeTexCoord')
tex_coord.location = (-700, 0)

# Noise texture for star positions — use Generated coords, fully static
noise = nodes.new('ShaderNodeTexNoise')
noise.location = (-450, 0)
noise.inputs['Scale'].default_value = 900.0
noise.inputs['Detail'].default_value = 0.0
noise.inputs['Roughness'].default_value = 0.0
noise.inputs['Distortion'].default_value = 0.0
links.new(tex_coord.outputs['Generated'], noise.inputs['Vector'])

# Sharp threshold — only top peaks become stars
ramp = nodes.new('ShaderNodeValToRGB')
ramp.location = (-220, 0)
ramp.color_ramp.interpolation = 'CONSTANT'
ramp.color_ramp.elements[0].position = 0.0
ramp.color_ramp.elements[0].color = (0, 0, 0, 1)
ramp.color_ramp.elements[1].position = 0.84
ramp.color_ramp.elements[1].color = (1, 1, 1, 1)
links.new(noise.outputs['Fac'], ramp.inputs['Fac'])

# Mix dark sky with star emission based on mask
mix = nodes.new('ShaderNodeMixShader')
mix.location = (400, 0)
links.new(ramp.outputs['Color'], mix.inputs['Fac'])
```

```
links.new(mix.outputs[0], output.inputs['Surface'])

# Dark sky background
bg_dark = nodes.new('ShaderNodeBackground')
bg_dark.location = (150, 100)
bg_dark.inputs['Color'].default_value = (0.005, 0.005, 0.025, 1)
bg_dark.inputs['Strength'].default_value = 1.0
links.new(bg_dark.outputs[0], mix.inputs[1])

# Star glow — slightly warm white
bg_stars = nodes.new('ShaderNodeBackground')
bg_stars.location = (150, -100)
bg_stars.inputs['Color'].default_value = (1.0, 1.0, 0.9, 1)
bg_stars.inputs['Strength'].default_value = 4.0
links.new(bg_stars.outputs[0], mix.inputs[2])

# No animation on any world nodes — fully static stars
# Clear any leftover animation data
if world.node_tree.animation_data:
    world.node_tree.animation_data_clear()

print("Rebuilt sky with Generated coordinates — fully static stars, no animation. Shoulc

```

13 Code Block #13 1670 bytes

```

import bpy

scene = bpy.context.scene

# --- Enable compositor for bloom/glare effect ---
scene.use_nodes = True
comp_nodes = scene.node_tree.nodes
comp_links = scene.node_tree.links

# Clear default compositor nodes
for node in comp_nodes:
    comp_nodes.remove(node)

# Render Layers input
render_layers = comp_nodes.new('CompositorNodeRLayers')
render_layers.location = (-200, 300)

# Glare node for bloom — makes emissions glow beautifully
glare = comp_nodes.new('CompositorNodeGlare')
```

```
glare.location = (200, 300)
glare.glare_type = 'FOG_GLOW'  # soft bloom
glare.quality = 'HIGH'
glare.threshold = 0.8           # only bright emissions bloom
glare.size = 7                  # medium glow spread

comp_links.new(render_layers.outputs['Image'], glare.inputs['Image'])

# Composite output
composite = comp_nodes.new('CompositorNodeComposite')
composite.location = (500, 300)
comp_links.new(glare.outputs['Image'], composite.inputs['Image'])

# Also add viewer node for preview
viewer = comp_nodes.new('CompositorNodeViewer')
viewer.location = (500, 100)
comp_links.new(glare.outputs['Image'], viewer.inputs['Image'])

# --- Optimize EEVEE settings ---
eevee = scene.eevee
eevee.taa_render_samples = 64       # good quality for final render
eevee.taa_samples = 32              # better viewport preview

print("Compositor bloom added (Fog Glow, threshold 0.8)")
print("EEVEE samples: 64 render / 32 viewport")
print("")
print("To see the full effect:")
print("  - Press F12 to do a single-frame test render")
print("  - Or switch viewport shading to 'Rendered' mode (Z menu → Rendered)")
print("  - For best quality, you could switch to Cycles (slower but gorgeous)")

```

14 Code Block #14 1579 bytes

```

import bpy

scene = bpy.context.scene

# --- Enable compositor for bloom/glare effect ---
scene.use_nodes = True
tree = scene.node_tree
comp_nodes = tree.nodes
comp_links = tree.links

# Clear default compositor nodes
```

```
for node in comp_nodes:
    comp_nodes.remove(node)

# Render Layers input
render_layers = comp_nodes.new('CompositorNodeRLayers')
render_layers.location = (-200, 300)

# Glare node for bloom — makes emissions glow beautifully
glare = comp_nodes.new('CompositorNodeGlare')
glare.location = (200, 300)
glare.glare_type = 'FOG_GLOW'  # soft bloom
glare.quality = 'HIGH'
glare.threshold = 0.8           # only bright emissions bloom
glare.size = 7                  # medium glow spread

comp_links.new(render_layers.outputs['Image'], glare.inputs['Image'])

# Composite output
composite = comp_nodes.new('CompositorNodeComposite')
composite.location = (500, 300)
comp_links.new(glare.outputs['Image'], composite.inputs['Image'])

# Also add viewer node for preview
viewer = comp_nodes.new('CompositorNodeViewer')
viewer.location = (500, 100)
comp_links.new(glare.outputs['Image'], viewer.inputs['Image'])

# --- Optimize EEVEE settings ---
eevee = scene.eevee
eevee.taa_render_samples = 64       # good quality for final render
eevee.taa_samples = 32              # better viewport preview

print("Compositor bloom added (Fog Glow, threshold 0.8)")
print("EEVEE samples: 64 render / 32 viewport")
print("")
print("To see the full effect:")
print("  - Press F12 to do a single-frame test render")
print("  - Or switch viewport shading to 'Rendered' mode (Z key)")

```

15 Code Block #15 589 bytes ✓ Executed

```python

import bpy

# Check how to access compositor in Blender 5.1
scene = bpy.context.scene

# Try different ways to access compositor
print("scene.use_nodes:", hasattr(scene, 'use_nodes'))
print("scene.node_tree:", hasattr(scene, 'node_tree'))

# Check if compositor is accessed differently
for attr in dir(scene):
    if 'node' in attr.lower() or 'comp' in attr.lower():
        print(f"  scene.{attr}")

# Check EEVEE attributes
eevee = scene.eevee
for attr in dir(eevee):
    if 'bloom' in attr.lower() or 'sample' in attr.lower() or 'taa' in attr.lower():
        print(f"  eevee.{attr}")

```

16 Code Block #16 1709 bytes

```python

import bpy

scene = bpy.context.scene

# --- Enable compositor ---
scene.use_nodes = True

# In Blender 5.1, compositor is accessed via compositing_node_group
# If none exists, we need to create one
comp_group = scene.compositing_node_group
if comp_group is None:
    # Create a new compositor node group
    comp_group = bpy.data.node_groups.new("Compositing Nodetree", 'CompositorNodeTree')
    scene.compositing_node_group = comp_group

comp_nodes = comp_group.nodes
comp_links = comp_group.links

# Clear existing nodes
for node in comp_nodes:
```

```
    comp_nodes.remove(node)

# Render Layers input
render_layers = comp_nodes.new('CompositorNodeRLayers')
render_layers.location = (-200, 300)

# Glare node for bloom
glare = comp_nodes.new('CompositorNodeGlare')
glare.location = (200, 300)
glare.glare_type = 'FOG_GLOW'
glare.quality = 'HIGH'
glare.threshold = 0.8
glare.size = 7

comp_links.new(render_layers.outputs['Image'], glare.inputs['Image'])

# Composite output
composite = comp_nodes.new('CompositorNodeComposite')
composite.location = (500, 300)
comp_links.new(glare.outputs['Image'], composite.inputs['Image'])

# Viewer node
viewer = comp_nodes.new('CompositorNodeViewer')
viewer.location = (500, 100)
comp_links.new(glare.outputs['Image'], viewer.inputs['Image'])

# --- Optimize EEVEE ---
eevee = scene.eevee
eevee.taa_render_samples = 64
eevee.taa_samples = 32

print("Bloom compositor set up with Fog Glow!")
print("EEVEE: 64 render / 32 viewport samples")
print("")
print("Try pressing F12 to render a test frame — you should see:")
print("  - Glowing bullet trails")
print("  - Bright muzzle flash")
print("  - Stars twinkling in the sky")
print("  - Soft bloom on all emissive surfaces")

```

17 Code Block #17 550 bytes ✓ Executed

```python

import bpy

# Inspect what attributes CompositorNodeGlare has in Blender 5.1
scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

# Create a temp glare node to inspect
glare = comp_group.nodes.new('CompositorNodeGlare')
for attr in sorted(dir(glare)):
    if not attr.startswith('_') and not attr.startswith('bl_'):
        try:
            val = getattr(glare, attr)
            if not callable(val):
                print(f"  {attr} = {val}")
        except:
            pass

comp_group.nodes.remove(glare)

```

18 Code Block #18 515 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

glare = comp_group.nodes.new('CompositorNodeGlare')

# Check inputs — Blender 5.1 moved settings to node inputs
print("INPUTS:")
for i, inp in enumerate(glare.inputs):
    print(f"  [{i}] '{inp.name}' type={inp.type} default={getattr(inp, 'default_value',

print("\nOUTPUTS:")
for i, out in enumerate(glare.outputs):
    print(f"  [{i}] '{out.name}' type={out.type}")

comp_group.nodes.remove(glare)

```

19 Code Block #19 729 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

# Check available menu options for the Type input
glare = comp_group.nodes.new('CompositorNodeGlare')
type_input = glare.inputs['Type']
print(f"Type input class: {type_input.__class__.__name__}")
print(f"Type rna: {type_input.rna_type.name}")

# Check enum items
for prop in type_input.rna_type.properties:
    print(f"  prop: {prop.identifier} = {prop.type}")
    if hasattr(prop, 'enum_items'):
        for item in prop.enum_items:
            print(f"    enum: {item.identifier} = {item.name}")

# Try listing available values
print(f"\nCurrent default_value: {type_input.default_value}")

comp_group.nodes.remove(glare)

```

20 Code Block #20 437 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

# Check enum options for the Glare type
glare = comp_group.nodes.new('CompositorNodeGlare')
type_input = glare.inputs['Type']

# Get the enum items for default_value
prop = type_input.rna_type.properties['default_value']
for item in prop.enum_items:
    print(f"  '{item.identifier}' — {item.name}")

comp_group.nodes.remove(glare)

```

21 Code Block #21 779 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

glare = comp_group.nodes.new('CompositorNodeGlare')

# Try setting type to various options
for val in ['Fog Glow', 'FOG_GLOW', 'Bloom', 'BLOOM', 'Ghosts', 'Simple Star', 'Streaks'
    try:
        glare.inputs['Type'].default_value = val
        print(f"  '{val}' — OK! Current: {glare.inputs['Type'].default_value}")
    except Exception as e:
        print(f"  '{val}' — FAILED: {e}")

# Also check Quality menu
for val in ['High', 'HIGH', 'Medium', 'Low']:
    try:
        glare.inputs['Quality'].default_value = val
        print(f"  Quality '{val}' — OK!")
    except Exception as e:
        print(f"  Quality '{val}' — FAILED: {e}")

comp_group.nodes.remove(glare)

```

22 Code Block #22 1503 bytes

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

nodes = comp_group.nodes
links = comp_group.links

# Clear existing compositor nodes
for node in nodes:
    nodes.remove(node)

# Render Layers input
```

```
render_layers = nodes.new('CompositorNodeRLayers')
render_layers.location = (-200, 300)

# Glare node — Fog Glow for soft bloom on emissions
glare = nodes.new('CompositorNodeGlare')
glare.location = (200, 300)
glare.inputs['Type'].default_value = 'Fog Glow'
glare.inputs['Quality'].default_value = 'High'
glare.inputs['Threshold'].default_value = 0.7      # catch bright emissions
glare.inputs['Size'].default_value = 0.6            # soft spread
glare.inputs['Strength'].default_value = 1.0

links.new(render_layers.outputs['Image'], glare.inputs['Image'])

# Composite output
composite = nodes.new('CompositorNodeComposite')
composite.location = (500, 300)
links.new(glare.outputs['Image'], composite.inputs['Image'])

# Viewer node for backdrop preview
viewer = nodes.new('CompositorNodeViewer')
viewer.location = (500, 100)
links.new(glare.outputs['Image'], viewer.inputs['Image'])

# --- EEVEE quality settings ---
eevee = scene.eevee
eevee.taa_render_samples = 64
eevee.taa_samples = 32

print("Bloom compositor ready! (Fog Glow, threshold 0.7, high quality)")
print("EEVEE: 64 render / 32 viewport samples")
print("")
print("Press F12 to render a test frame — bullets, muzzle flash,")
print("and stars will all have a beautiful soft glow!")

```

23 Code Block #23 611 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

# List available compositor node types
print("Available compositor node types:")
for item in bpy.types.CompositorNode.__subclasses__():
    name = item.__name__
    if 'compos' in name.lower() or 'output' in name.lower() or 'render' in name.lower()
        print(f"  {name}")

# Try finding the right names
print("\nAll node types with 'Comp' or 'Out' or 'View':")
for item in bpy.types.CompositorNode.__subclasses__():
    print(f"  {item.__name__}")

```

24 Code Block #24 896 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group

# Check what's already in the default compositor
print("Existing nodes in compositor group:")
for node in comp_group.nodes:
    print(f"  '{node.name}' type={node.type} bl_idname={node.bl_idname}")
    for i, inp in enumerate(node.inputs):
        print(f"    in[{i}]: '{inp.name}' linked={inp.is_linked}")
    for i, out in enumerate(node.outputs):
        print(f"    out[{i}]: '{out.name}' linked={out.is_linked}")

print("\nExisting links:")
for link in comp_group.links:
    print(f"  {link.from_node.name}:{link.from_socket.name} → {link.to_node.name}:{link

# Check interface (group inputs/outputs)
print("\nGroup interface items:")
for item in comp_group.interface.items_tree:
    print(f"  {item.name} — {item.item_type} in_out={getattr(item, 'in_out', 'N/A')}")

```

25 Code Block #25 813 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group
nodes = comp_group.nodes
links = comp_group.links

# Try creating a Group Output node (generic node type)
try:
    group_out = nodes.new('NodeGroupOutput')
    print(f"NodeGroupOutput created: {group_out.name}")
except Exception as e:
    print(f"NodeGroupOutput failed: {e}")

# Also try to find all creatable node types
print("\nTrying common output node types:")
for ntype in ['NodeGroupOutput', 'CompositorNodeOutputFile', 'CompositorNodeViewer',
              'CompositorNodeOutput', 'NodeOutput', 'CompositorNodeSplitViewer']:
    try:
        n = nodes.new(ntype)
        print(f"  '{ntype}' — OK! name={n.name}")
        nodes.remove(n)
    except Exception as e:
        print(f"  '{ntype}' — {e}")

```

26 Code Block #26 1763 bytes ✓ Executed

```python

import bpy

scene = bpy.context.scene
scene.use_nodes = True
comp_group = scene.compositing_node_group
nodes = comp_group.nodes
links = comp_group.links

# Clear all existing nodes
for node in nodes:
    nodes.remove(node)

# Create the group output socket first (Image output)
interface = comp_group.interface
```

```python
# Clear existing interface items
for item in list(interface.items_tree):
    try:
        interface.remove(item)
    except:
        pass

# Add an Image output to the group
interface.new_socket(name="Image", in_out='OUTPUT', socket_type='NodeSocketColor')

# Render Layers
render_layers = nodes.new('CompositorNodeRLayers')
render_layers.location = (-200, 300)

# Glare — Fog Glow bloom
glare = nodes.new('CompositorNodeGlare')
glare.location = (200, 300)
glare.inputs['Type'].default_value = 'Fog Glow'
glare.inputs['Quality'].default_value = 'High'
glare.inputs['Threshold'].default_value = 0.7
glare.inputs['Size'].default_value = 0.6
glare.inputs['Strength'].default_value = 1.0

links.new(render_layers.outputs['Image'], glare.inputs['Image'])

# Group Output node
group_out = nodes.new('NodeGroupOutput')
group_out.location = (500, 300)
links.new(glare.outputs['Image'], group_out.inputs['Image'])

# Viewer node for backdrop preview
viewer = nodes.new('CompositorNodeViewer')
viewer.location = (500, 50)
links.new(glare.outputs['Image'], viewer.inputs['Image'])

# --- EEVEE quality ---
eevee = scene.eevee
eevee.taa_render_samples = 64
eevee.taa_samples = 32

print("Compositor bloom fully wired!")
print("  Render Layers → Glare (Fog Glow) → Group Output + Viewer")
print("  Threshold: 0.7 | Quality: High")
print("")
print("Press F12 to render a test frame — you'll see soft glow on")
print("bullets, muzzle flash, robot eyes, and stars!")

```